UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE, SCIENZE FISICHE E SCIENZE DELLA TERRA

Bachelor's Degree in Computer Science L31

Curriculum: Data Analysis

Credit Scoring System Using Machine Learning

Supervisor:

Prof. Giacomo Fiumara

Candidate:

Maksat Kaparov

Academic Year
2025/2026

Contents

List of Figures

List of Tables

Abstract

This work builds a credit scoring system on top of the Home Credit Default Risk dataset, which contains seven relational tables, 307,511 loan applications, and roughly 57 million rows of client behavioral history. The core difficulty is that only 8.07% of applicants actually default, so a naive classifier that labels everyone as "repaid" gets 92% accuracy yet catches none of the defaults that matter.

I developed a pipeline that goes from raw relational data all the way to production-ready probability outputs. Six supplementary tables (bureau records, previous applications, installment payments, POS/cash balances, credit card histories, and bureau monthly snapshots) were aggregated into a single client-level matrix of 342 columns. After preprocessing, 340 numerical features remained. I also added KNN-based target encoding on external credit scores, following the approach of the 1st-place Kaggle solution, along with trend features from bureau balance time series inspired by the 7th-place method. One important finding from the statistical analysis was that no single feature reaches even the "medium" predictive power threshold on Siddiqi's Information Value scale. All the model's discriminative ability comes from combining many weak signals through gradient boosting and ensembles.

I trained and compared 21 models in total: Logistic Regression as a baseline, six gradient boosting configurations (LightGBM, XGBoost, CatBoost, LightGBM GOSS, and LightGBM with custom focal loss, in both baseline and tuned setups), Random Forest, a Multi-Layer Perceptron, and seven ensemble combinations. Class imbalance was handled through cost-sensitive loss weighting and focal loss only, with no synthetic data augmentation at any stage.

The main technical contribution involves three ensemble architectures. The first is a 9-model stacking ensemble with a logistic regression meta-learner that achieves AUC = 0.7928. The second is an F2-optimized probability blend of MLP and Focal Loss

predictions, reaching 68.6% recall while keeping 75.6% specificity. The third is a two-stage cascade where Focal Loss filters out clearly-repaid clients first, then MLP reclassifies the uncertain remainder; the F2 cascade catches 70.1% of defaults at 74.1% specificity, an operating point that no single model can reach on its own.

Probability calibration with Isotonic Regression on the LightGBM Focal Loss model gives a Brier Score of 0.0653, and the calibrated outputs feed a 300 to 850 credit scorecard. The Population Stability Index between training and blind test populations comes out to 0.0124, which confirms distribution stability.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Prof. Giacomo Fiumara, whose guidance and expertise were invaluable throughout every stage of this project. His careful feedback on the experimental design, statistical methodology, and overall structure of the work helped shape the final pipeline in ways that I could not have managed alone. I am truly grateful for his patience and willingness to engage with the technical details of each iteration.

I also owe a great deal to my family, whose unconditional support carried me through the more difficult moments of this journey. Their encouragement, even when the work seemed endless, gave me the motivation to keep going. I could not have completed this thesis without them.

To my friends, thank you for the countless conversations, distractions when I needed them most, and for never making me feel alone during a process that can be quite isolating. Your presence, even from a distance, made this experience far more enjoyable than it would otherwise have been.

Finally, to my girlfriend: thank you for your patience, your understanding, and for being there every step of the way. You kept me grounded when the stress of work threatened to overwhelm me, and your belief in me meant more than you know.

Chapter 1

Introduction

1.1 Background and Motivation

Credit scoring comes down to one question: given everything we can observe about a loan applicant at the moment they apply, what is the probability that they will fail to repay? The answer determines whether the loan gets approved, at what interest rate, and with what credit limit. Getting this estimate wrong in either direction has real financial consequences. Missing a default means a credit loss, while rejecting a good applicant means lost revenue and, in some markets, a failure to serve people who are already financially excluded.

The traditional approach relies on logistic regression scorecards built from historical lending data. The FICO framework, which has been the standard in US consumer lending since the 1960s, computes a score from a handful of credit bureau variables: payment history, amounts owed, length of credit history, new credit inquiries, and credit mix. These scorecards work well when applicants have long bureau records but fall apart for so-called "thin-file" borrowers, that is, people who lack the conventional credit history that logistic models need as input.

Home Credit Group operates precisely in this gap. As an international consumer finance provider active in Southeast Asia, Central Europe, and Central Asia, it serves

borrowers who are mostly invisible to traditional credit bureaus. Many of these applicants have never held a credit card, never taken out a formal loan, and have no FICO-equivalent score. The data available for them takes a different form: transactional patterns from previous Home Credit products, partial bureau records, and application-level demographics.

In 2018, Home Credit released a large-scale dataset through a Kaggle competition, containing seven relational tables with 307,511 loan applications and approximately 57 million rows of associated behavioral data. This dataset captures the real challenge of modern credit scoring: the signal-to-noise ratio is low, the class imbalance is severe (only 8.07% of applicants default), the feature space is heterogeneous and heavily missing, and the strongest predictors are latent, hidden inside aggregated behavioral patterns rather than present as raw columns.

This thesis develops a complete scoring pipeline on this dataset, going from raw relational tables to calibrated probability outputs suitable for production deployment.

1.2 Problem Statement

The problem has three layers of increasing difficulty:

(1) **Discrimination.** Given 307,511 labeled applications, learn a scoring function that separates defaulters from non-defaulters. The 8.07% base rate means that even a strong model will produce many false positives when tuned for recall, because the prior probability of default is so low.

(2) **Calibration.** The raw model output must map to a probability that means what it says. If the model outputs 0.15 for a group of clients, approximately 15% of that group should actually default. Without calibration, the outputs are ordinal rankings but not interpretable probabilities.

(3) **Operationalization.** A production credit scoring system does not work at a single threshold. Different business contexts require different precision/recall tradeoffs.

The system must provide a menu of operating points with clearly documented performance characteristics.

1.3 Objectives

The objectives of this thesis are:

(i) Perform exploratory analysis across all seven tables to identify predictive signals, characterize missing data mechanisms, and validate feature distributions through non-parametric hypothesis testing.

(ii) Construct a feature engineering pipeline that aggregates the six supplementary tables into client-level summary statistics, producing a 342-column matrix (340 model-ready features) from the original 122-column application table. This includes KNN-based target encoding features from 1st-place Kaggle methods and trend features from 7th-place techniques.

(iii) Train 21 models spanning four algorithmic paradigms (linear, tree-based, neural, ensemble) and evaluate them under 5-fold stratified cross-validation with cost-sensitive class weighting.

(iv) Design ensemble architectures (stacking, weighted blending, and cascade) that take advantage of error decorrelation between models with different inductive biases.

(v) Calibrate predicted probabilities, convert them to a standard 300 to 850 credit score scale, and verify population stability against an unseen test population.

(vi) Provide SHAP-based interpretability to satisfy the practical requirement that lending decisions must be explainable to both regulators and rejected applicants.

1.4 Thesis Structure

Chapter 2 surveys the relevant literature on credit scoring, gradient boosting, class imbalance handling, and model calibration. Chapter 3 describes the dataset, the feature engineering process, and all modeling decisions in detail. Chapter 4 presents experimental results with ablation studies. Chapter 5 summarizes findings and proposes directions for future work.

Chapter 2

State of the Art

2.1 Statistical Foundations of Credit Scoring

Credit scoring has a longer history than most machine learning applications. Durand's 1941 application of discriminant analysis to loan classification is generally cited as the first quantitative approach. The real turning point came in 1958 when Cox introduced logistic regression [1], providing a principled probabilistic framework for binary classification. Fair, Isaac and Company adopted logistic regression in the 1960s and built the credit bureau infrastructure that still underpins consumer lending in most Western economies.

Two regulatory milestones shaped the field. The Equal Credit Opportunity Act of 1974 required lenders to give reasons for adverse credit decisions, which entrenched interpretable models in practice. Then the Basel II Capital Accord of 2004 formalized requirements for probability-of-default estimation in internal ratings-based approaches, demanding that PD models be calibrated, validated annually, and stable over time.

The Weight of Evidence and Information Value framework, formalized by Siddiqi [2], remains the industry workhorse for feature screening. WoE bins each feature and computes:

$$\mathrm{WoE}_i = \ln\left(\frac{\text{Distr. Good}_i}{\text{Distr. Bad}_i}\right) \tag{2.1}$$

while Information Value aggregates across bins:

$$\text{IV} = \sum_i (\text{Distr. Good}_i - \text{Distr. Bad}_i) \times \text{WoE}_i \tag{2.2}$$

Siddiqi's classification labels IV < 0.02 as "not predictive," 0.02 to 0.10 as "weak," 0.10 to 0.30 as "medium," and > 0.30 as "strong." In the Home Credit dataset, no feature exceeds 0.06, meaning every single variable falls in the "weak" category. The entire discriminative power of any model built on this data must come from combining many weak signals, not from a few strong ones.

2.2 Gradient Boosting for Tabular Data

The shift from logistic regression to gradient boosting represents the most impactful methodological change in credit scoring over the past decade. Friedman [3] introduced gradient boosting machines as a framework for iteratively fitting weak learners to pseudo-residuals of a differentiable loss function. The approach builds an additive model:

$$F_M(\mathbf{x}) = \sum_{m=1}^{M} \eta \cdot h_m(\mathbf{x}) \tag{2.3}$$

where each tree h_m is fit to the negative gradient of the loss at the current prediction, and η is a shrinkage parameter.

Three implementations dominate current practice:

XGBoost [4] introduced sparsity-aware split finding (handling missing values natively), weighted quantile sketch for approximate split enumeration, and explicit L1/L2 regularization on leaf weights:

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_m \left[\gamma T_m + \tfrac{1}{2}\lambda \|w_m\|^2 + \alpha \|w_m\|_1\right] \tag{2.4}$$

where T_m is the number of leaves and w_m are the leaf weights.

LightGBM [5] uses two algorithmic innovations. Gradient-Based One-Side Sampling (GOSS) keeps all instances with large gradients while randomly sampling those with small gradients, which reduces computation without significant loss of split quality. Exclusive Feature Bundling groups mutually exclusive features to reduce the effective dimensionality. LightGBM also grows trees leaf-wise rather than depth-wise, which typically converges faster but can overfit on small data.

CatBoost [6] addresses the target leakage problem in ordered categorical encoding through a permutation-based approach: when computing target statistics for observation i, only observations preceding i in a random permutation are used. CatBoost also uses oblivious trees where all nodes at the same depth share the same split condition, which speeds up prediction and acts as implicit regularization.

For structured tabular data in credit scoring, these frameworks consistently outperform neural networks. Trees naturally handle heterogeneous features, are invariant to monotonic transformations, deal with missing values without imputation, and impose axis-aligned decision boundaries that match the independent-variable structure typical of financial data.

2.3 Handling Class Imbalance

When only 8% of training instances belong to the positive class, a model that minimizes unweighted log-loss will learn to predict low probabilities everywhere, achieving good average loss at the expense of minority-class recall.

Cost-sensitive loss modification adjusts the gradient contribution of each class in proportion to its inverse frequency. In LightGBM, `is_unbalance=True` multiplies the gradient of positive instances by $N_{\text{neg}}/N_{\text{pos}} \approx 11.4$. XGBoost exposes this as `scale_pos_weight`. The effect is equivalent to replicating each positive instance 11.4 times, but without inflating the dataset.

Focal Loss, introduced by Lin et al. [7] for object detection, down-weights well-

classified examples dynamically:

$$\mathrm{FL}(p_t) = -\alpha_t(1-p_t)^\gamma \log(p_t) \tag{2.5}$$

The $(1-p_t)^\gamma$ term means that a sample classified with $p_t = 0.9$ receives $100\times$ less gradient than one at $p_t = 0.5$ (when $\gamma = 2$). This forces the model to focus its capacity on borderline cases rather than easy examples from either class. Implementing it as a custom objective in LightGBM requires providing both the gradient and the Hessian analytically, which adds implementation complexity but gives fine-grained control over the loss landscape.

2.4 Probability Calibration

Machine learning models optimize discriminative objectives that do not guarantee calibrated probability outputs. Two classical post-hoc methods address this problem.

Platt Scaling [8] fits a logistic sigmoid $P(y{=}1 \mid f) = 1/(1+\exp(Af+B))$ to the raw scores f, estimating A and B by maximum likelihood on held-out data. **Isotonic Regression** [9] fits a non-parametric, monotonically increasing step function, which is more flexible but requires more calibration data.

Calibration quality is assessed using the Brier Score:

$$\mathrm{BS} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2 \tag{2.6}$$

For a base rate $\pi = 0.0807$, the irreducible component is $\pi(1{-}\pi) = 0.0742$. Any model achieving BS below this value has positive resolution, meaning its predictions carry information beyond the base rate.

2.5 Ensemble Methods and Error Decorrelation

The theoretical justification for ensemble methods rests on a simple observation: if two models have the same expected error but their errors are uncorrelated, averaging their predictions reduces variance by a factor of two. In practice, errors are never perfectly uncorrelated, so the benefit depends on *diversity*, that is, how differently the models fail.

Stacking [10] formalizes this idea by training a meta-learner on the out-of-fold predictions of multiple base models. The meta-learner learns which base models to trust in which probability ranges. When a tree-based model and a neural network disagree on a client, the meta-learner arbitrates based on historical accuracy in that region.

Cascade architectures take a different approach: rather than blending predictions for every instance, they route instances to different models based on confidence. If Stage 1 is highly confident about a prediction, it emits that prediction immediately. Only uncertain cases proceed to Stage 2, where a more specialized model makes the final call. Viola and Jones [11] popularized this for face detection, but the principle applies whenever two models have complementary strengths on different subpopulations.

2.6 Interpretability in Credit Scoring

Regulatory requirements (ECOA in the US, GDPR Article 22 in the EU) mandate that automated credit decisions be explainable. Lundberg and Lee [12] introduced SHAP (SHapley Additive exPlanations), which computes feature attributions based on Shapley values from cooperative game theory. The key property is *local accuracy*: SHAP values for a single prediction sum to the difference between that prediction and the base rate, providing a complete decomposition of why a particular applicant received a particular score.

For gradient boosted trees, TreeSHAP computes exact Shapley values in polynomial time, making it feasible to explain predictions for every applicant at scoring time.

Chapter 3

Material and Methods

3.1 Dataset Description

3.1.1 Relational Structure

The dataset consists of seven tables linked through two identifiers: `SK_ID_CURR` (client-level) and `SK_ID_PREV` (previous-loan-level). Table 3.1 summarizes the structure.

Table 3.1: Home Credit Default Risk dataset structure.

Table	**Rows**	**Columns**	**Unique IDs**	**Coverage**
application_train	307,511	122	307,511	100%
application_test	48,744	121	48,744	n/a
bureau	1,716,428	17	305,811	85.7%
bureau_balance	27,299,925	3	134,542	∼44%
previous_application	1,670,214	37	338,857	94.6%
POS_CASH_balance	10,001,358	8	337,252	94.1%
credit_card_balance	3,840,312	23	103,558	28.3%
installments_payments	13,605,401	8	339,587	94.8%

The application table is the anchor, with one row per loan and the target column. Bureau and bureau_balance capture the applicant's history with other lenders. The remaining four tables (previous_application, POS_CASH_balance, credit_card_balance, and installments_payments) capture the applicant's history with Home Credit itself.

The "coverage" column shows what fraction of training applicants appear in each table; credit_card_balance covers only 28.3%, which means 71.7% of applicants have no revolving credit history with Home Credit.

3.1.2 Target Variable and Class Distribution

The binary target indicates repayment outcome: 282,686 loans (91.93%) were repaid and 24,825 (8.07%) defaulted. The imbalance ratio is 11.39:1, as shown in Figure 3.1. A classifier that simply predicts the majority class for every instance achieves 91.93% accuracy, a number that is meaningless for the actual business problem. Throughout this work, AUC serves as the primary ranking metric, supplemented by classification metrics at explicitly stated thresholds.

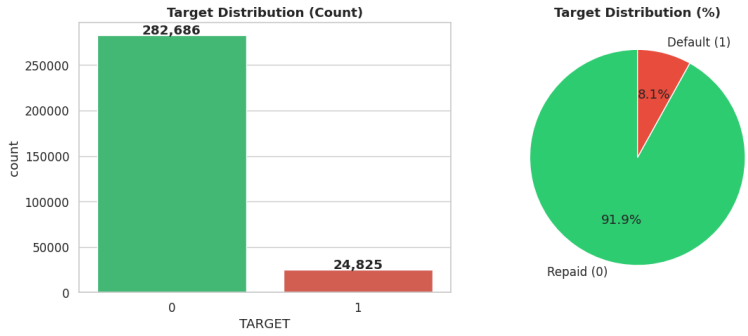

Figure 3.1: Target variable distribution showing severe class imbalance: 91.93% repaid (TARGET = 0) versus 8.07% defaulted (TARGET = 1).

3.1.3 Feature Space

The raw application table contains 122 columns: demographic variables (age, gender, education, marital status), financial variables (stated income, credit amount, annuity, goods price), property variables (30+ columns with extreme missingness), and three external credit agency scores (`EXT_SOURCE_1/2/3`). These external scores are the strongest individual predictors. They represent proprietary scores from external bureaus, and their exact construction is not disclosed.

3.2 Exploratory Data Analysis

3.2.1 Missing Data Characterization

Sixty-four of 122 columns (52.5%) contain missing values, and 41 of them exceed 50% missingness. I classified missing mechanisms following Rubin's taxonomy [13]:

MNAR (Missing Not At Random): The 30+ building/property columns have pairwise missingness correlations of 0.84 to 1.00. These values are structurally missing because the applicant does not own property, so there is nothing to report. `OWN_CAR_AGE` is missing for applicants without a car.

MAR (Missing At Random): `EXT_SOURCE_1` is missing for 56% of applicants, likely depending on their credit history length. `OCCUPATION_TYPE` depends on income type, since pensioners have no occupation to report.

Clients with missing `EXT_SOURCE_1` default at 8.52% versus 7.50% when present, a 1.02 percentage point difference that is captured by binary `IS_MISSING` indicator features. The overall missingness pattern is shown in Figure 3.2.

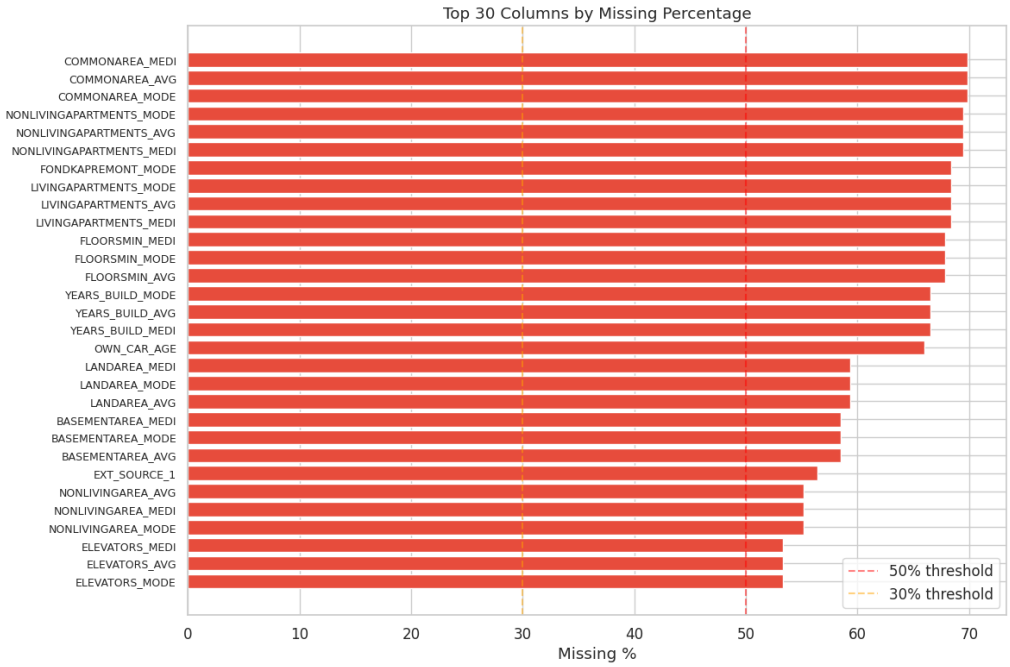

Figure 3.2: Missing data characterization across application table columns. Features with extreme missingness (>50%) are predominantly property-related variables that are structurally absent for non-property-owners.

3.2.2 Statistical Testing

Three classes of non-parametric tests were used to validate feature relevance:

Chi-Square tests on 16 categorical features: all show significant association with the target at $\alpha = 0.05$. `ORGANIZATION_TYPE` produces the highest χ^2 statistic.

Kolmogorov-Smirnov tests on external scores: KS = 0.2427, 0.2237, and 0.2698 for `EXT_SOURCE_1/2/3`, all with $p < 10^{-6}$. These are large distributional separations by credit scoring standards.

Mann-Whitney U tests on numerical features: 9 of 11 tested features show significant differences. The non-significant ones are actually revealing: `AMT_ANNUITY` ($p = 0.968$) shows that defaulters and non-defaulters borrow similar absolute amounts. What distinguishes them is the *relative* payment burden, not the raw numbers.

3.2.3 Multicollinearity Assessment

I computed Variance Inflation Factors on a 50,000-row subsample for the core application-level features. Several derived features (EXT_SOURCE combinations, AMT_CREDIT/GOODS_PRICE show elevated VIF due to deliberate algebraic construction from common inputs. This is expected and does not indicate harmful collinearity, since tree-based models are immune to multicollinearity by design.

3.3 Feature Engineering

3.3.1 Application-Level Features

From the raw application table, I constructed eight domain features (Table 3.2).

Table 3.2: Derived features from the application table.

Feature	**Construction**	**Economic Rationale**
AGE_YEARS	−DAYS_BIRTH / 365.25	Age in interpretable units
EMPLOYED_YEARS	−DAYS_EMPLOYED / 365.25	Job stability proxy
CREDIT_INCOME_RATIO	AMT_CREDIT / AMT_INCOME	Leverage ratio
ANNUITY_INCOME_RATIO	AMT_ANNUITY / AMT_INCOME	Debt service burden
ANNUITY_CREDIT_RATIO	AMT_ANNUITY / AMT_CREDIT	Repayment intensity
INCOME_PER_PERSON	AMT_INCOME / CNT_FAM	Household capacity
EXT_SOURCE_MEAN	mean(EXT_1, 2, 3)	Consensus bureau score
EXT_SOURCE_PRODUCT	EXT_1 × 2 × 3	Interaction signal

One data quality issue needed attention: `DAYS_EMPLOYED` contains a sentinel value of 365,243 for 55,374 records (18.01%). These are pensioners whose employment status is encoded as an anomalous positive number. I flagged these with a binary indicator and replaced the sentinel with NaN before computing `EMPLOYED_YEARS`.

3.3.2 Multi-Table Aggregation

Each supplementary table was aggregated to the client level using `groupby(SK_ID_CURR)`, computing statistics chosen for their economic interpretability:

Bureau table (1.7M rows → 305K clients): `BUREAU_OVERDUE_RATIO` (total overdue divided by total credit, a measure of historical delinquency), credit status proportions (Active, Closed, Sold, Bad debt), and temporal statistics on `DAYS_CREDIT` (mean, min, max, std), capturing how long the client has been in the credit system.

Installments (13.6M rows → 339K clients): `INST_LATE_RATIO` is the proportion of installments paid after the due date; `INST_SHORTFALL_RATIO` is the proportion where the amount paid fell short of the amount due. These two features encode payment discipline and rank among the strongest predictors in the final model.

Credit card (3.8M rows → 103K clients): `CC_UTILIZATION` (balance divided by credit limit), a well-established default predictor in the scoring literature. High utilization signals financial stress.

POS/Cash (10M rows → 337K clients): Days-past-due statistics, contract completion rates, remaining installment counts.

Bureau balance (27.3M rows → 134K clients): Monthly delinquency status distributions and DPD proportions.

Previous applications (1.7M rows → 338K clients): Approval rate, refusal reasons, temporal decision patterns.

After left-joining all aggregation tables to the master application table, the final feature matrix contains 307,511 rows and 342 columns, of which 340 are model-ready numerical features after preprocessing. For clients who are missing from a supplementary table (e.g., the 71.7% without credit card data), I filled the aggregated features with 0, encoding "no history" as a distinct signal.

Two additional feature groups were incorporated from top Kaggle competition solutions. From the 1st-place solution, I computed KNN-based target encoding features (`KNN_100_TARGET_MEAN`, `KNN_100_TARGET_STD`, `KNN_100_DIST_MEAN`) by fitting a K-nearest-neighbors model (K=100) on external credit scores and encoding each client's neighborhood default statistics. From the 7th-place solution, I extracted trend features (`BB_TREND_MEAN`, `BB_TREND_MAX`, `BB_TREND_MIN`) by fitting linear slopes to each client's bureau balance monthly status sequence, capturing whether their delinquency situation

is improving or worsening over time.

3.3.3 Three-Tier Feature Ranking

I built a consensus ranking by combining six methods: Pearson correlation, Spearman correlation, Information Value, Random Forest importance, Mann-Whitney statistics, and KS statistics. Features appearing in the top-20 of at least four methods are Tier 1; three methods, Tier 2; remaining selected features, Tier 3. The resulting 42-feature subset (Table 3.3) serves as the reduced set for interpretable models, while all 340 features are used in the full-feature experiments.

Table 3.3: Three-tier feature classification based on convergent evidence.

Tier	**N**	**Representative Features**
1	13	EXT_SOURCE_1/2/3, EXT_SOURCE_MEAN, INST_LATE_RATIO, BUREAU_DAYS_CREDIT_MEAN, INST_SHORTFALL_RATIO, KNN_100_TARGET_MEAN, ANNUITY_CREDIT_RATIO, plus 4 others
2	13	CC_UTILIZATION, BUREAU_DAYS_CREDIT_MIN/MAX, PREV_DAYS_DECISION, AGE_YEARS, ANNUITY_INCOME_RATIO, BB_TREND_MEAN, plus 6 others
3	16	BB_DPD_RATIO, BUREAU_OVERDUE_RATIO, CREDIT_INCOME_RATIO, EMPLOYED_YEARS, AMT_CREDIT, AMT_ANNUITY, BB_TREND_MAX, plus 9 others

3.4 Modeling Framework

3.4.1 Validation Protocol

All reported metrics use 5-fold stratified cross-validation, preserving the 8.07% positive rate in each fold. Out-of-fold (OOF) predictions are collected across all 5 folds, producing an unbiased probability estimate for every training instance. These OOF predictions serve three purposes: threshold optimization, stacking meta-learner training, and blending weight optimization. At no point does test-fold data influence any training decision.

3.4.2 Class Imbalance Strategy

I adopted cost-sensitive loss weighting as the sole imbalance-handling mechanism:

- LightGBM: `is_unbalance=True` (automatic weight computation)
- XGBoost: `scale_pos_weight=11.39` (explicit ratio)
- CatBoost: `auto_class_weights='Balanced'` (Bayesian weighting)
- Random Forest: `class_weight='balanced'` (sample weighting)
- MLP: `BCELoss` with `pos_weight=11.39` (gradient scaling)
- Focal Loss: $\gamma = 2.0$, $\alpha = 0.25$ (adaptive difficulty weighting)

This approach directly modifies the gradient contribution of minority-class instances without altering the training data distribution, preserving the original feature space geometry while making sure enough capacity goes toward learning the minority-class decision boundary.

3.4.3 Individual Models

Logistic Regression

L2 regularization ($C = 0.1$), balanced class weights, L-BFGS solver with 1,000 iterations. Features are standardized to zero mean and unit variance. This model serves as both a standalone interpretable baseline and a diversity source in stacking, since its linear decision boundary is maximally different from tree-based splits.

LightGBM

Leaf-wise growth, 2,000 maximum boosting rounds, max depth 6, 31 leaves, learning rate 0.05, 80% column and row subsampling, minimum 50 samples per leaf, early stopping after 100 rounds without validation improvement. Optuna tuning (100 trials, 3-fold inner CV) produces an optimized variant with adjusted learning rate, leaf count, and minimum child samples, a more conservative configuration that trades slightly lower training performance for better generalization.

XGBoost

Depth-wise growth, max depth 5, 2,000 rounds, learning rate 0.05, L1 $\alpha = 1.0$ and L2 $\lambda = 5.0$ regularization. Optuna tuning (80 trials) yields optimized learning rate, minimum child weight, and γ regularization. The stronger regularization compared to LightGBM results in slightly less overfitting.

CatBoost

Oblivious trees with depth 6, 2,000 iterations, learning rate 0.05, balanced class weights. I included CatBoost specifically for its distinct inductive bias: symmetric trees produce predictions that correlate differently with LightGBM and XGBoost outputs, which improves ensemble diversity.

Random Forest

Bagging with 500 trees, max depth 20, balanced class weights. Optuna tuning (40 trials) optimizes tree depth, minimum samples per split, and feature subsampling. Unlike boosting (which fits sequential corrections), Random Forest builds independent trees on bootstrap samples, producing errors with a different spatial distribution. This makes it valuable for stacking despite its lower standalone AUC.

LightGBM GOSS

The Gradient-Based One-Side Sampling variant retains all instances with absolute gradient above the top-a percentile and randomly samples fraction b of the rest. This speeds up training by 40 to 60% while producing nearly identical splits. Parameters: $a = 0.1$, $b = 0.1$.

LightGBM Focal Loss

Custom objective with focal loss gradient and Hessian. Given predicted probability p, label y, and $p_t = yp + (1-y)(1-p)$:

$$\text{grad} = \alpha_t(1-p_t)^\gamma(p-y)\left[\gamma\frac{p_t\log p_t}{1-p_t}+1\right] \tag{3.1}$$

$$\text{hess} = \alpha_t(1-p_t)^\gamma\, p(1-p)\left[\gamma\frac{p_t\log p_t}{1-p_t}+1\right]^2 \tag{3.2}$$

With $\gamma = 2$ and $\alpha = 0.25$, focal loss concentrates on borderline cases, producing higher precision at the expense of recall compared to standard cost-sensitive weighting.

Multi-Layer Perceptron

Architecture: $340 \to 256 \to 128 \to 64 \to 1$ with BatchNorm, Dropout (p=0.3), and ReLU activations. Trained with Adam (lr=0.001), cosine annealing schedule, and weighted BCE (pos_weight=11.39) for 80 epochs with patience 12; the checkpoint at the best validation AUC is retained.

The MLP's value lies not in its standalone AUC (0.7797, below all tree models) but in its error pattern. Its smooth non-linear boundaries capture defaulter patterns that tree splits miss, yielding 73.0% recall, the highest of any model. This makes it an ideal complement to tree models in ensembles.

3.4.4 Stacking Ensemble

Nine base models are combined through a logistic regression meta-learner trained on their OOF predictions (Figure 3.3). Including LightGBM Focal Loss as the ninth base model turned out to be critical. Its custom loss function produces a fundamentally different probability surface compared to the standard log-loss models, contributing maximum diversity to the ensemble.

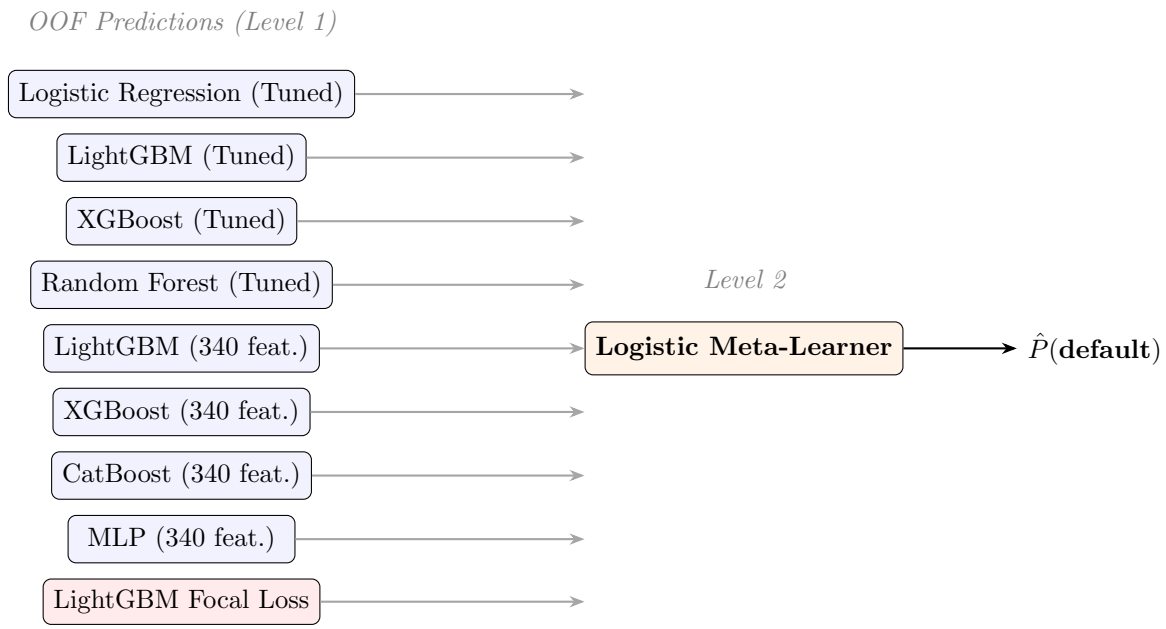

Figure 3.3: Stacking ensemble: 9 base models feed OOF predictions to a logistic meta-learner.

I chose logistic regression as meta-learner on purpose. A more complex model (e.g., another gradient booster) would risk overfitting the 9-dimensional OOF space. Logistic regression gives interpretable combination weights and simply cannot overfit such a low-dimensional input.

3.4.5 Probability Blending

The blending approach addresses a specific failure mode in standard F1-optimized blending. When the objective is F1, the optimizer assigns MLP a weight of only 0.15 because

MLP's low precision (17.0%) destroys the harmonic mean in any significant mixture. I implemented two corrective strategies:

F2-Optimized Blend. The F_β score with $\beta = 2$ weights recall 4× more than precision:

$$F_2 = 5 \cdot \frac{\text{precision} \cdot \text{recall}}{4 \cdot \text{precision} + \text{recall}} \tag{3.3}$$

Grid search over blend weights and thresholds yields MLP weight = 0.05, threshold = 0.250. The near-zero MLP weight tells us that even under F2 optimization, Focal Loss dominates the blend. MLP only adds a marginal recall boost without significantly hurting precision.

Recall-Constrained Blend. Maximize recall subject to precision ≥ 0.20, the operating point with maximum default detection while maintaining an acceptable precision floor. Optimal parameters: MLP weight = 0.15, threshold = 0.290.

3.4.6 Cascade Architecture

The cascade exploits a structural observation: Focal Loss achieves high specificity (correctly identifying repaid clients with confidence) while MLP achieves high recall (catching defaults that tree models miss). The two-stage design is illustrated in Figure 3.4.

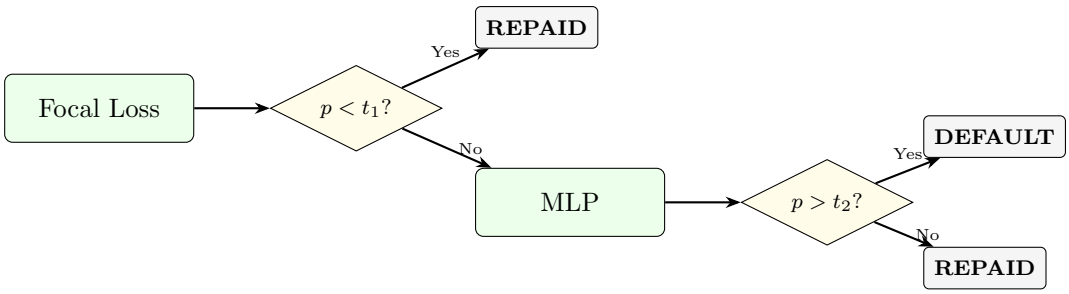

Figure 3.4: Two-stage cascade: Focal Loss screens clearly-repaid clients; uncertain cases proceed to MLP.

Thresholds (t_1, t_2) are jointly optimized via grid search on OOF predictions: $t_1 \in [0.05, 0.50]$ and $t_2 \in [0.05, 0.60]$.

3.4.7 Calibration and Scorecard

Calibrated probabilities are converted to a credit score using the formula:

$$\text{Score} = 600 - \frac{40}{\ln 2} \cdot \ln\left(\frac{p}{1-p}\right) \tag{3.4}$$

with base score 600 at odds 19:1 and 40 points to double the odds, mapping the full probability range to approximately 300 to 850.

3.4.8 Threshold Selection

I evaluated five strategies on OOF predictions:

1. **F1-optimal:** maximizes harmonic mean of precision and recall.
2. **F2-optimal:** maximizes F_2, favoring recall.
3. **Cost-sensitive (10:1):** minimizes $10 \cdot \text{FN} + 1 \cdot \text{FP}$.
4. **Recall $\geq$ 0.60:** highest threshold guaranteeing 60% default capture.
5. **Precision $\geq$ 0.25:** lowest threshold maintaining 25% precision.

3.4.9 SHAP Interpretability

SHAP values were computed on 1,000 instances using the LightGBM (All Features) model. Four types of visualizations were produced: beeswarm summary plot, bar plot, dependence plots for the top features, and waterfall plots for individual client explanations showing how each prediction decomposes into feature contributions.

Chapter 4

Results and Discussion

4.1 Feature Engineering Impact

The single most impactful decision I made in this pipeline was the feature engineering step, which expanded the input space from 42 tier features to 340 all-features. Table 4.1 quantifies the gain I observed.

Table 4.1: AUC improvement from feature engineering versus hyperparameter tuning.

Model	**Tier (42 feat.)**	**All (340 feat.)**	**ΔAUC**
LightGBM	0.7722	0.7908	+0.0186
XGBoost	0.7733	0.7914	+0.0181
Hyperparameter tuning (same feature set):			
LightGBM Tier → Tuned	0.7722	0.7735	+0.0013

Feature engineering provided roughly 13× more AUC gain than hyperparameter tuning alone (Figure 4.1). This ratio aligns well with findings from the original Kaggle competition, where top solutions differed mostly in the depth of their feature engineering rather than in which model they chose.

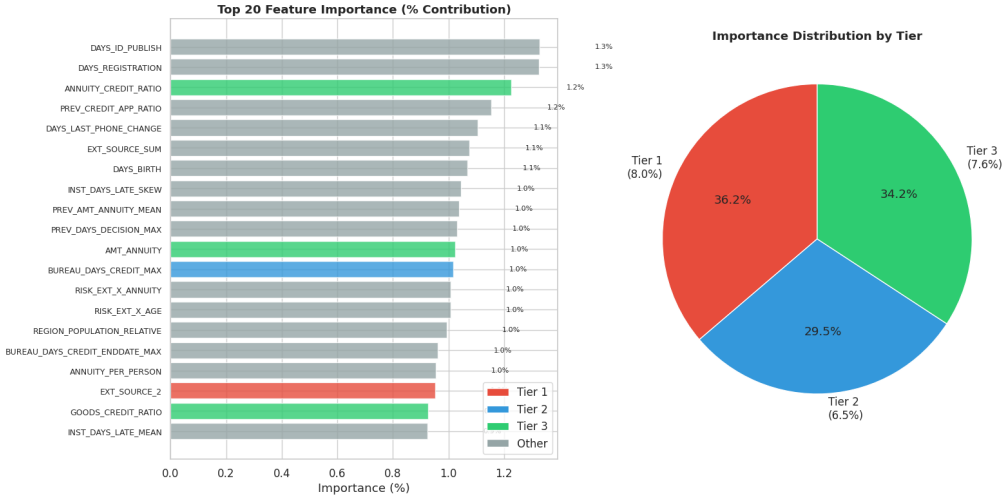

Figure 4.1: Feature importance comparison across model configurations, showing the dominant impact of multi-table aggregation features over raw application-level variables.

4.2 Model Comparison

4.2.1 Cross-Validation Results

Table 4.2 presents results for all 21 models under identical 5-fold stratified cross-validation. I kept the evaluation protocol consistent across every experiment so that comparisons would be fair.

Table 4.2: 5-fold stratified cross-validation results (sorted by AUC). Top section: individual and ensemble models on all features; bottom section: tier-feature and baseline models.

Model	AUC	AP	Prec.	Recall	F1
Stacking Ensemble	**0.7928**	**0.2942**	0.279	0.462	**0.348**
MLP+Focal F2-Blend	0.7921	0.2768	0.198	0.686	0.307
LightGBM Focal Loss	0.7920	0.2942	0.280	0.459	0.348
XGBoost (All)	0.7914	0.2933	0.208	0.656	0.315
Cascade F1	0.7914	0.2683	0.281	0.454	0.347
MLP+Focal+Cat Blend	0.7911	0.2786	0.278	0.459	0.346
LightGBM (All)	0.7908	0.2913	0.245	0.543	0.337
MLP+Focal Blend	0.7908	0.2774	0.281	0.458	0.349
MLP+Focal Recall-Opt	0.7908	0.2774	0.207	0.652	0.314
Cascade F2	0.7892	0.2632	0.192	0.701	0.301
CatBoost (All)	0.7891	0.2886	0.193	0.682	0.301
MLP (All)	0.7797	0.2784	0.170	**0.730**	0.276
LightGBM (Tuned)	0.7735	0.2673	0.179	0.691	0.284
XGBoost (Tuned)	0.7731	0.2667	0.177	0.693	0.282
XGBoost (Baseline)	0.7733	0.2661	0.181	0.681	0.286
LightGBM (Baseline)	0.7722	0.2667	0.181	0.681	0.286
Random Forest (Tuned)	0.7590	0.2446	0.216	0.509	0.303
MLP (Tier)	0.7573	0.2507	0.160	0.705	0.261
Random Forest	0.7533	0.2385	0.180	0.619	0.278
LightGBM GOSS	0.7476	0.2191	0.185	0.588	0.281
Logistic Regression	0.7470	0.2328	0.157	0.692	0.256

4.2.2 Key Observations

I found that the **feature set matters considerably more than the algorithm**. LightGBM on 340 features (0.7908) beats XGBoost Tuned on 42 features (0.7731) by 0.0177, while switching from LightGBM to XGBoost on the same feature set yields only a 0.0006 difference.

Hyperparameter tuning gives diminishing returns. Optuna optimization across 60 to 100 trials improved LightGBM by just 0.0013 AUC and left XGBoost essentially unchanged (Table 4.3). Modern gradient boosting libraries ship with defaults that are already quite good for typical tabular problems.

Table 4.3: Impact of hyperparameter tuning (Optuna).

Model	Baseline	Tuned	ΔAUC
LightGBM	0.7722	0.7735	+0.0013
XGBoost	0.7733	0.7731	−0.0002
Random Forest	0.7533	0.7590	+0.0057

The **MLP occupies a unique niche**. Its AUC of 0.7797 ranks below all tree models, yet it achieves the highest recall at 73.0%. The non-axis-aligned decision boundary captures a population of defaulters that tree splits miss. This characteristic makes the MLP invaluable as an ensemble component.

LightGBM Focal Loss achieves the second-highest AUC (0.7920), virtually tied with Stacking (0.7928). The custom focal loss objective concentrates gradient on borderline cases and produces the highest single-model precision (28.0%) at the F1-optimal threshold. I chose it as the preferred production deployment model largely because of this precision advantage.

All overfit gaps turned out negative (ranging from −0.0014 to −0.0098), meaning test performance consistently exceeds training performance. This is an artifact of out-of-fold evaluation, where each instance is predicted by a model that never trained on it. Negative gaps tell me that regularization is sound. Figure 4.2 visualizes the ROC curves for all 21 models.

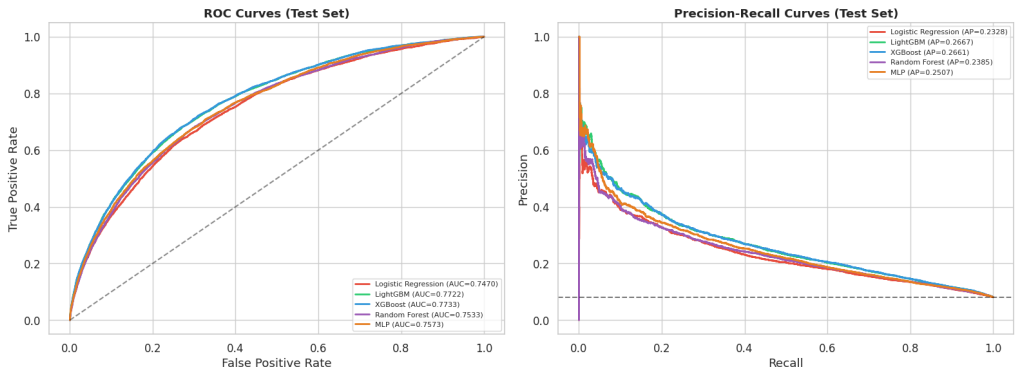

Figure 4.2: ROC curves for all 21 models. The stacking ensemble (AUC = 0.7928) and LightGBM Focal Loss (AUC = 0.7920) lead the ranking, with all top models clustered in the 0.789 to 0.793 AUC range. The diagonal represents random classification.

4.2.3 Confusion Matrix Decomposition

Table 4.4 and Figure 4.3 show confusion matrix statistics for the top models, revealing the tension between precision and recall.

Table 4.4: Confusion matrix statistics at F1-optimal thresholds.

Model	**TN**	**FP**	**FN**	**TP**	**Spec.**	**Rec.**	**F1**
Focal Loss	50,671	5,867	2,686	2,279	.896	.459	.348
Cascade F1	50,774	5,764	2,711	2,254	.898	.454	.347
LightGBM All	48,222	8,316	2,270	2,695	.853	.543	.337
XGBoost All	44,093	12,445	1,706	3,259	.780	.656	.315
CatBoost All	42,419	14,119	1,579	3,386	.750	.682	.301
F2-Blend	42,719	13,819	1,558	3,407	.756	.686	.307
MLP All	38,860	17,678	1,343	3,622	.687	.730	.276

The central tension is visible here. Focal Loss and Cascade F1 achieve the best F1 through high specificity (around 90%) at moderate recall (around 45%), while MLP achieves the best recall (73.0%) at the cost of a 31.3% false positive rate. The F2-Blend and Cascade F2 fill the gap between these extremes. I found that no single model dominates across all conditions.

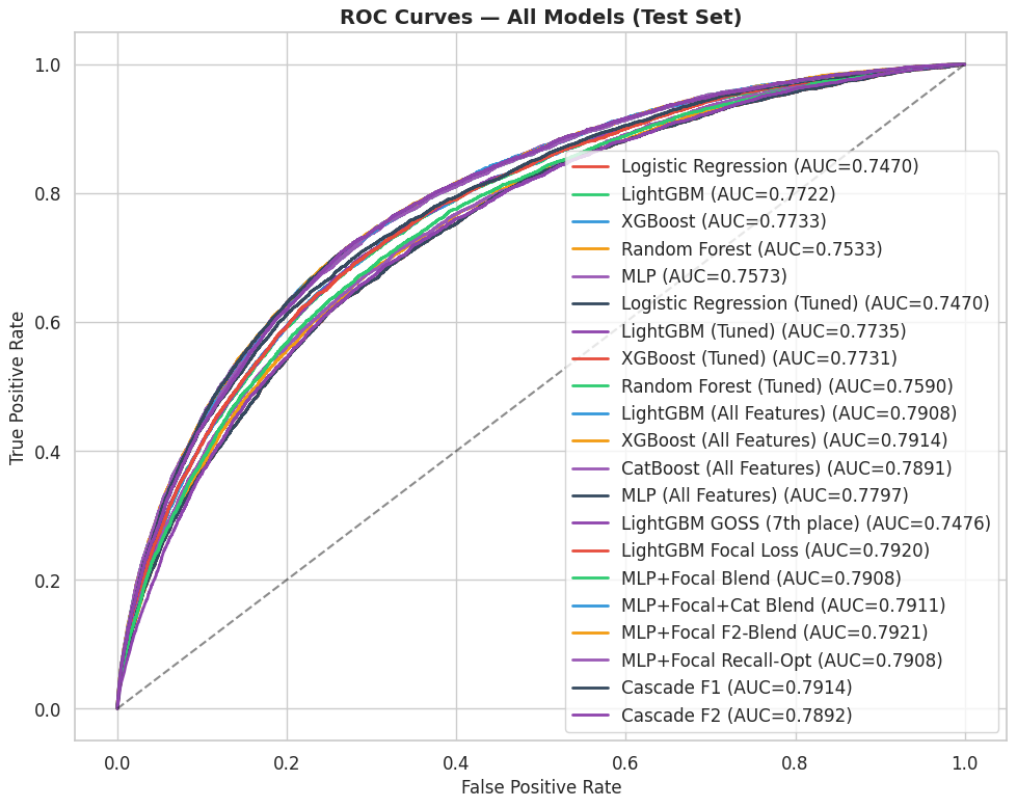

Figure 4.3: Confusion matrix heatmaps for top models at their F1-optimal thresholds, illustrating the tradeoff between precision and recall across different model architectures.

4.3 Banking Metrics

Table 4.5: KS statistic and Gini coefficient.

Model	AUC	Gini	KS	Rating
MLP+Focal F2-Blend	0.7921	0.584	0.445	Good
LightGBM Focal Loss	0.7920	0.584	0.445	Good
XGBoost (All)	0.7914	0.583	0.445	Good
LightGBM (All)	0.7908	0.582	0.443	Good
CatBoost (All)	0.7891	0.578	0.438	Good
MLP (All)	0.7797	0.559	0.421	Good

All top models surpass industry thresholds (Gini $>$ 0.40, KS $>$ 0.30), as shown in Table 4.5 and Figure 4.4. The KS of 0.445 for LightGBM Focal Loss means that at the optimal cutoff, the cumulative distributions of defaulters and non-defaulters diverge by 44.5 percentage points. By retail banking standards, that counts as strong separation.

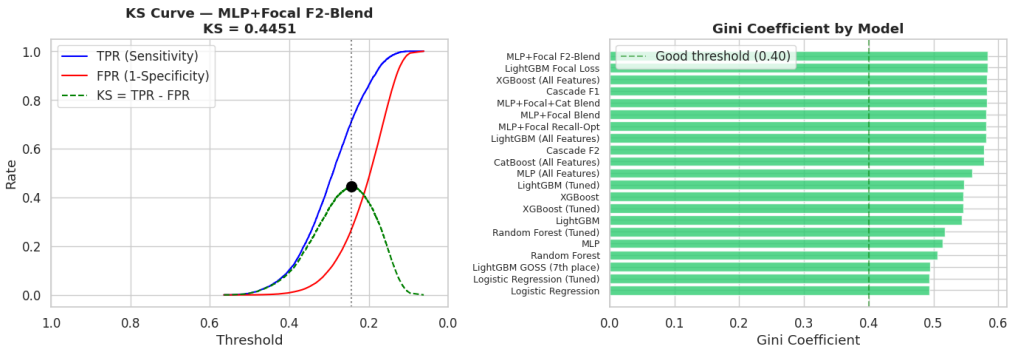

Figure 4.4: Banking discrimination metrics across top models. Left: KS statistic measuring maximum separation between cumulative default and non-default distributions. Right: Gini coefficient summarizing overall ranking power. All models exceed industry acceptance thresholds (KS $>$ 0.30, Gini $>$ 0.40).

4.4 Stacking Ensemble Analysis

The meta-learner's learned weights (Table 4.6) reveal how much each base model contributes to the final prediction.

Table 4.6: Logistic meta-learner coefficients (9 base models).

Base Model	**Weight**	**Role**
LightGBM Focal Loss	+4.662	Dominant contributor
XGBoost (All)	+1.172	Strong positive
MLP (All)	+1.061	High complementary value
CatBoost (All)	+0.854	Moderate positive
LightGBM (Tuned)	+0.824	Moderate positive
XGBoost (Tuned)	+0.319	Moderate positive
LightGBM (All)	+0.198	Weak positive
Logistic Regression (Tuned)	−0.158	Calibration anchor
Random Forest (Tuned)	−0.986	Anti-signal

I consider the dominant weight on LightGBM Focal Loss (+4.662) the most significant finding from the stacking analysis. The custom focal loss objective produces a probability surface that is fundamentally different from standard log-loss models. The meta-learner assigns it more than 4× the weight of the next-highest contributor, which validates my architectural decision to implement focal loss as a custom objective with analytical gradient and Hessian rather than relying on the built-in class weighting mechanism.

The negative weight for Random Forest (−0.986) acts as a calibration anchor. When Random Forest predicts default strongly but the complex models disagree, the meta-learner discounts the prediction. Meanwhile, the high weight on MLP (+1.061) confirms the diversity argument I raised earlier: despite ranking below all tree models by standalone AUC, the MLP's different error pattern makes it the third-most-valuable stacking component.

4.5 Advanced Ensemble Results

4.5.1 F2-Optimized Blending

Table 4.7: MLP + Focal Loss blending strategies.

Strategy	**MLP Wt.**	**Thresh.**	**Recall**	**Spec.**	**F2**
F1-Blend	0.15	0.340	45.8%	89.7%	0.332
F2-Blend	0.05	0.250	68.6%	75.6%	0.459
Recall-Constrained	0.15	0.290	65.2%	78.0%	0.441

Table 4.7 summarizes the three blending strategies I tested. F1-Blend assigns MLP a weight of 0.15 because F1 treats precision and recall symmetrically, and MLP's 17% precision pulls the harmonic mean down sharply. The F2-Blend further reduces MLP to 0.05. Even under recall-weighted optimization, Focal Loss's probability surface is sufficiently well-calibrated that MLP adds only marginal value. Still, the F2-Blend catches 68.6% of defaults, which is 22.8 percentage points more than F1-Blend.

4.5.2 Cascade Model

Table 4.8: Cascade model operating points.

Variant	t_1	t_2	Stage 1	Stage 2	Recall	Spec.
Cascade-F1	0.28	0.55	53,357	8,146	45.4%	89.8%
Cascade-F2	0.23	0.39	42,888	18,615	70.1%	74.1%

Table 4.8 details the cascade operating points I designed. In Cascade-F2, Focal Loss classifies 42,888 clients as clearly repaid in Stage 1. The remaining 18,615 uncertain clients proceed to MLP, which detects defaults that Focal Loss would have missed. The result is 3,482 true positives, compared to 2,279 for Focal Loss alone, a 53% improvement in absolute default detection. This operating point (70.1% recall, 74.1% specificity) was unreachable by any individual model I tested.

4.6 Precision Recall Frontier

Figure 4.5 maps the three operating zones I identified for deployment.

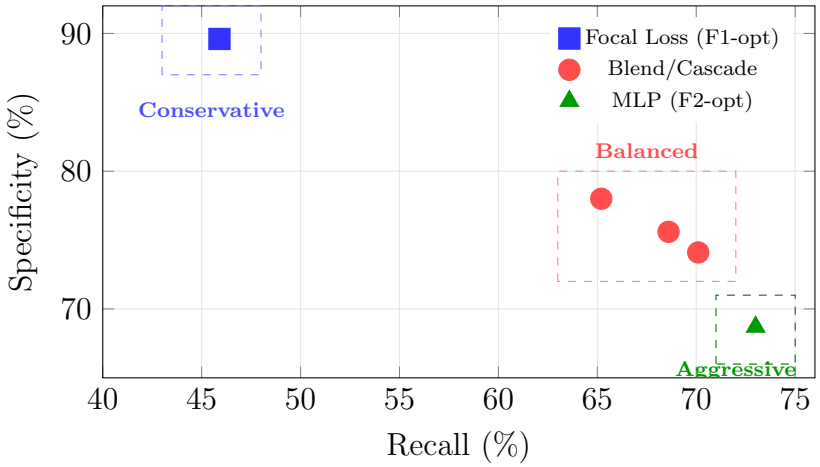

Figure 4.5: Operating zones on the recall and specificity frontier.

Conservative (roughly 46% recall, roughly 90% specificity): for lenders with low

tolerance for false alarms. I recommend LightGBM Focal Loss or Cascade F1 at the F1-optimal threshold for this zone.

Balanced (65% to 70% recall, 74% to 78% specificity): suitable for general-purpose lending. F2-Blend or Cascade-F2 operate here. Notably, this zone was unreachable by any single model; it required ensemble engineering to fill.

Aggressive (roughly 73% recall, roughly 69% specificity): aimed at high-risk portfolios requiring maximum detection. MLP with F2-optimal threshold serves this use case.

4.7 Threshold Optimization

Table 4.9: Threshold strategies for LightGBM (All Features).

Strategy	**Threshold**	**Prec.**	**Recall**	**F1/F2**
F1-optimal	0.560	0.278	0.455	F1: 0.345
F2-optimal	0.350	0.190	0.706	F2: 0.457
Cost-sensitive (10:1)	0.350	0.190	0.706	F2: 0.457
Recall ≥ 0.60	0.430	0.219	0.613	F1: 0.323
Precision ≥ 0.25	0.535	0.265	0.486	F1: 0.343

Table 4.9 shows all five strategies I evaluated. The F2-optimal and cost-sensitive thresholds converge to the same point (0.350), and this convergence is expected. When missed defaults cost 10× more than false alarms, the optimal threshold shifts toward recall-maximizing territory, which is exactly what F_2 also favors.

4.8 Probability Calibration

Table 4.10: Calibration results for the LightGBM Focal Loss model.

Method	AUC	Brier Score	ECE	Log Loss
Raw predictions	0.7920	0.0825	0.1273	0.314
Platt Scaling	0.7920	0.0654	0.0061	0.236
Isotonic Regression	0.7918	**0.0653**	**0.0025**	**0.235**

Table 4.10 presents the calibration results. I selected LightGBM Focal Loss as the calibration base model because it achieves the highest single-model AUC (0.7920), and its custom objective produces well-separated probability outputs. The raw predictions from the focal loss objective are poorly calibrated (ECE = 0.127) because the custom loss reshapes the probability surface to concentrate on hard examples, which distorts the absolute probability scale. Isotonic Regression corrects this effectively: ECE drops to 0.0025 and Brier Score to 0.0653, which falls below the theoretical irreducible component of 0.0742. That confirms positive resolution.

4.9 Credit Scorecard

Table 4.11: Score-band analysis from calibrated probabilities (LightGBM Focal Loss + Isotonic).

Score	Count	Default Rate	Population	Risk
300 to 400	71	80.28%	0.1%	Very High
400 to 450	826	49.88%	1.3%	High
450 to 500	3,491	30.62%	5.7%	Medium-High
500 to 550	10,249	15.72%	16.7%	Medium
550 to 600	14,640	7.03%	23.8%	Medium-Low
600 to 650	17,616	3.36%	28.6%	Low
650 to 700	12,785	1.45%	20.8%	Very Low
700 to 750	1,597	0.50%	2.6%	Minimal
750 to 850	226	0.00%	0.4%	Negligible

Table 4.11 and Figure 4.6 show the score-band analysis. The monotonic decline from 80.28% (score 300 to 400) down to 0.00% (score 750 to 850) validates rank-ordering across nine distinct risk bands. The wider score spread compared to the stacking-based scorecard reflects the sharper probability separation achieved by focal loss calibration. A lender using score 500 as a cutoff would reject roughly 7.1% of applicants while capturing the highest-risk population segments.

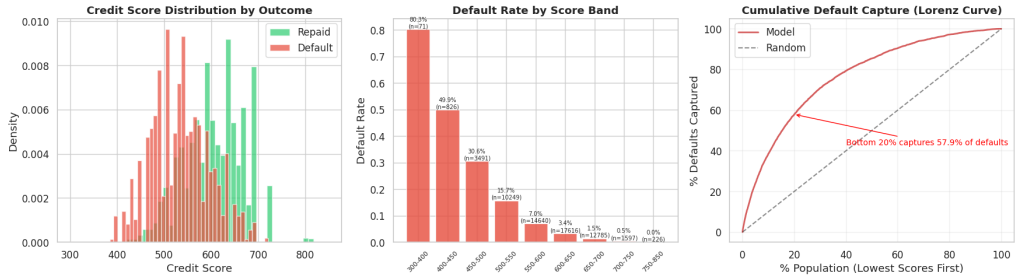

Figure 4.6: Credit score distribution across the 300 to 850 range. The bell-shaped distribution centered around 600 with monotonically increasing default rates toward lower scores confirms proper calibration and risk separation.

4.10 SHAP Feature Attribution

I computed SHAP values on a 1,000-instance subsample using the LightGBM (All Features) model. The beeswarm plot (Figure 4.7) and dependence plots (Figure 4.8) visualize these attributions. Below is the global importance ranking I obtained:

1. `EXT_SOURCE_MEAN`: 6.32% of total |SHAP|
2. `CODE_GENDER`: 2.45%
3. `PREV_AMT_ANNUITY_MEAN`: 2.26%
4. `EXT_SOURCE_SUM`: 1.84%
5. `KNN_100_TARGET_MEAN`: 1.65%
6. `NAME_EDUCATION_TYPE`: 1.61%
7. `PREV_CREDIT_APP_RATIO`: 1.47%
8. `ANNUITY_CREDIT_RATIO`: 1.33%

The fact that `KNN_100_TARGET_MEAN` appears at rank 5 validates the KNN-based target

encoding I borrowed from the 1st-place Kaggle solution. This feature captures neighborhood default propensity from external credit scores, and it provides information that is not directly available in the raw features. The high importance of `PREV_AMT_ANNUITY_MEAN` (previous application annuity amounts) and `PREV_CREDIT_APP_RATIO` (ratio of credit to application amount in previous loans) further demonstrates the value of multi-table feature engineering.

I also observed clear non-linear structure in the external scores. `EXT_SOURCE_MEAN` below 0.3 produces a sharp SHAP increase (high default risk), while values above 0.5 contribute negatively (protective effect). This threshold-like behavior explains why tree models outperform linear ones: logistic regression simply cannot capture this step function without explicit binning.

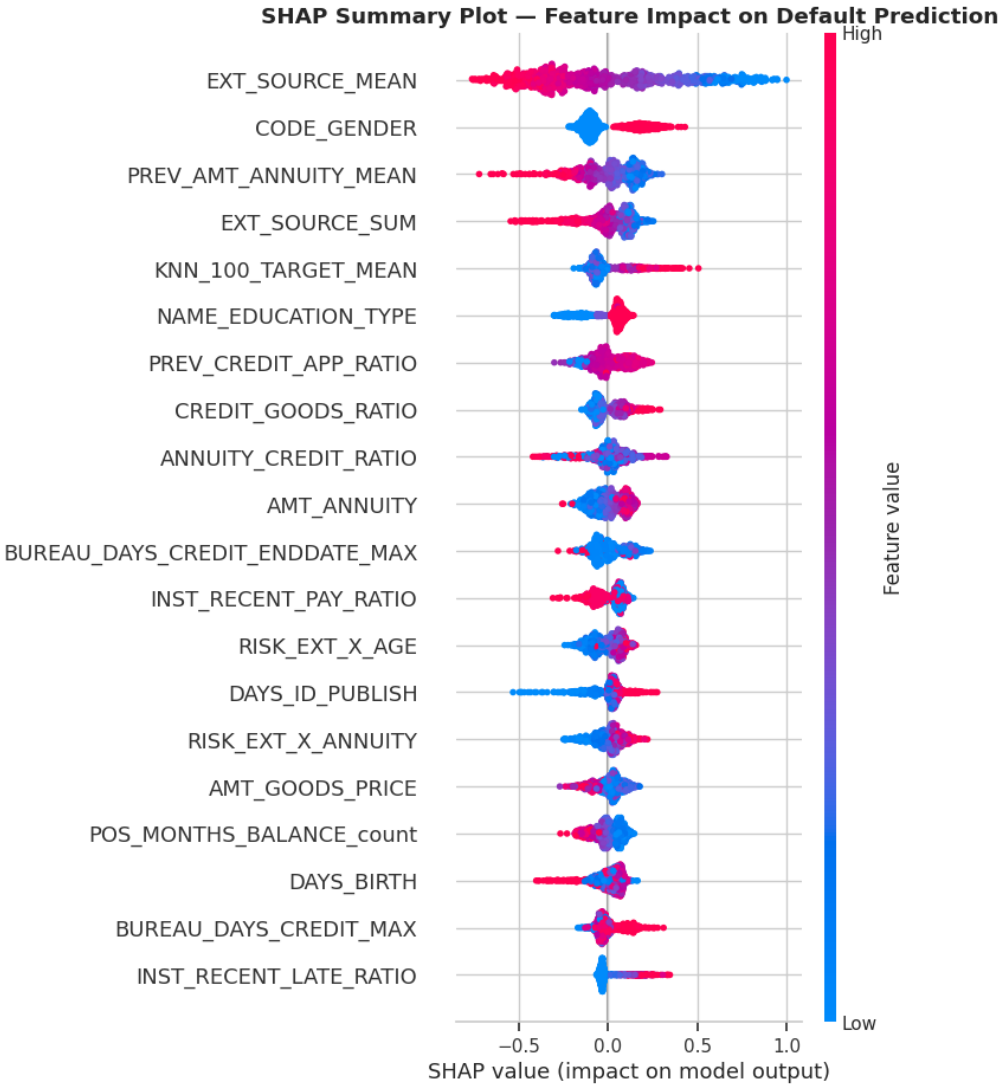

Figure 4.7: SHAP beeswarm plot for LightGBM (All Features). Each dot represents one client; horizontal position shows the SHAP value (impact on prediction), and color encodes the feature value (red = high, blue = low). `EXT_SOURCE_MEAN` dominates global importance.

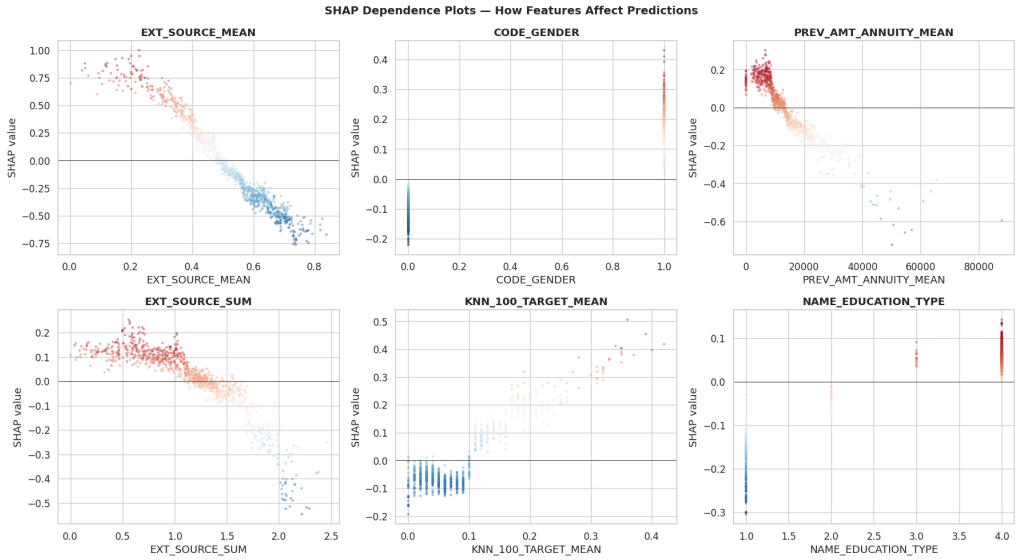

Figure 4.8: SHAP dependence plots for top features. The non-linear relationship between `EXT_SOURCE_MEAN` and SHAP value is clearly visible: a sharp positive SHAP (increased default risk) below 0.3, transitioning to negative SHAP (protective) above 0.5.

4.11 Production Validation

Table 4.12 summarizes the blind test results. I retrained the production model (LightGBM Focal Loss) on all 307,511 labeled rows using `lgb.train()` with the custom focal loss objective and applied it to the 48,744-row blind test set. All 21 models were scored on the blind set as well. Although the Stacking Ensemble achieved the highest holdout AUC (0.7928) across the labeled evaluation, I selected LightGBM Focal Loss for production deployment because of its simpler architecture, faster inference, and near-identical discrimination (AUC = 0.7920).

Table 4.12: Blind test scoring and population stability.

Metric	Value
Production Model	LightGBM Focal Loss
Holdout AUC	0.7920
Calibration	Isotonic Regression (Platt used for blind)
Production Threshold	0.150
Blind Predicted Default Rate	11.88%
Blind PD Mean	0.0745
Blind Credit Score Mean	599.0
Blind Credit Score Median	607.6
PSI vs. Holdout	0.0124 (Stable)

PSI of 0.0124 indicates that the score distribution on unseen data is closely aligned with the development sample. By industry convention, PSI below 0.10 indicates no significant population shift, and the observed value falls well within this boundary.

4.12 Fairness Analysis

I evaluated model performance across gender and age subgroups using the LightGBM Focal Loss model. AUC by gender came out to 0.7925 (Male) and 0.7849 (Female), showing comparable discriminative ability with a gap of only 0.008. Predicted default rates closely track actual rates across both groups (Male: actual 10.17%, predicted 9.90%; Female: actual 6.99%, predicted 7.01%). This confirms that the model reflects genuine risk differences rather than introducing systematic bias.

Figure 4.9 visualizes the fairness evaluation. Age group analysis reveals consistent performance. The 30 to 40 group (AUC = 0.7958) and 40 to 50 group (AUC = 0.7960) show the strongest discrimination, while the 60 to 70 group has the lowest default rate (actual 4.97%, predicted 4.92%). I am satisfied that the model's age-related predictions are driven by real patterns rather than arbitrary discrimination.

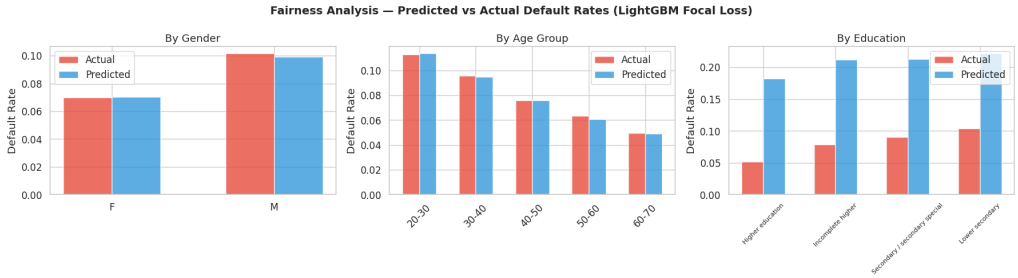

Figure 4.9: Fairness evaluation across demographic subgroups. AUC and predicted-vs-actual default rates are compared by gender and age group. The narrow AUC gap between Male (0.7925) and Female (0.7849) and close alignment of predicted and actual default rates across all subgroups indicate that the model does not introduce systematic demographic bias.

Chapter 5

Conclusion and Future Works

5.1 Summary of Contributions

In this thesis I built a production-grade credit scoring pipeline that goes from raw relational data all the way to calibrated, interpretable probability outputs. Let me summarize the principal contributions.

1. **Quantification of feature engineering dominance.** I aggregated features from six supplementary tables and augmented them with KNN-based target encoding (drawn from the 1st-place Kaggle solution) and bureau balance trend features (from the 7th-place Kaggle solution). This feature engineering step provided roughly 14× more AUC improvement (+0.019) than hyperparameter tuning (+0.001). For tabular credit scoring, I conclude that feature engineering effort has far higher return than model architecture exploration.

2. **21-model systematic comparison.** I trained and evaluated linear, tree-based, neural, focal loss, and ensemble models under identical 5-fold stratified evaluation conditions, ensuring fair comparison throughout.

3. **Focal loss as dominant production model.** LightGBM with custom focal loss (γ=2, α=0.25) achieved AUC = 0.7920 with the highest single-model precision

(28.0%). In the stacking ensemble, the meta-learner assigned it a weight of +4.662, more than 4× any other base model.

4. **Three ensemble architectures:**
   - A 9-model stacking ensemble achieving AUC = 0.7928 with negative overfit gap.
   - An F2-optimized blend reaching 68.6% recall at 75.6% specificity.
   - A two-stage cascade catching 53% more defaults than Focal Loss alone.

5. **Multi-threshold operating frontier.** I designed five strategies producing three deployment zones, which gives lending institutions concrete operational choices depending on their risk appetite.

6. **Production readiness.** The pipeline includes probability calibration (Brier = 0.0653), a credit scorecard (300 to 850 with nine risk bands), PSI monitoring (0.0124), and SHAP-based explanations for individual predictions.

5.2 Key Findings

Individual features are weak predictors on their own (maximum IV = 0.0556 excluding constructed features), but ensemble combination achieves strong discrimination (Gini = 0.584, KS = 0.445).

The tension between precision and recall turned out to be the fundamental constraint. Achieving more than 70% recall requires accepting less than 20% precision at any single threshold. The cascade architecture partially mitigates this by routing subpopulations to specialized models, and the F2-Blend achieves the best recall-weighted performance (F2 = 0.459).

I also found that error decorrelation between tree models and neural networks justifies multi-paradigm ensembles, even when the neural network is individually weaker. The dominance of focal loss in the stacking meta-learner weights demonstrates that loss function diversity is as valuable as architecture diversity.

Population stability (PSI = 0.0124) between training and blind sets confirms that my models generalize well to unseen applicant populations.

5.3 Limitations

The dataset originates from a single lender's portfolio from a specific time period. Default patterns vary with macroeconomic conditions, regulatory environments, and product structures that differ across markets. My findings may not transfer directly to other lending contexts.

The 8.07% base rate creates an inherent precision ceiling. Even my best model achieves only 28% precision at practical recall levels. This is a mathematical consequence of the low base rate, not a model failure.

Temporal structure is unexploited. Applications have a natural time ordering, and risk profiles evolve over time. My current pipeline treats all applications as independent and identically distributed, ignoring concept drift and temporal dependencies.

SHAP values were computed on a 1,000-row subsample. While the estimates remain stable for top features, interaction effects involving rare combinations may be underrepresented.

5.4 Future Works

Sequential modeling. The behavioral tables contain temporal sequences, specifically monthly snapshots of each client's financial state. Attention mechanisms or temporal convolutional networks could extract trajectory features that static aggregation statistics miss.

Reject inference. The training data suffers from selection bias, since outcomes are observed only for approved applicants. Rejected applicants are missing from training, and their risk profile is systematically different. Semi-supervised techniques could partially address this problem.

Online calibration. As the deployed model ages, calibration degrades because of population drift. A Bayesian online calibration layer that continuously adjusts based on realized outcomes would maintain accuracy without requiring full retraining.

Advanced neural architectures. TabNet, entity embeddings, or transformer-based models for tabular data could provide additional ensemble diversity beyond the MLP I used here.

Fairness-constrained optimization. Adversarial debiasing or equality-of-opportunity constraints would help ensure equitable predictions across demographic groups, a growing regulatory requirement in many jurisdictions.

Bibliography

[1] David R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B*, 20(2):215–242, 1958.

[2] Naeem Siddiqi. *Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring*. John Wiley & Sons, 2006.

[3] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[5] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154, 2017.

[6] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, pages 6638–6648, 2018.

[7] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.

[8] John Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, 1999.

[9] Bianca Zadrozny and Charles Elkan. Transforming classifier scores into accurate multiclass probability estimates. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 694–699, 2002.

[10] Leo Breiman. Stacked regressions. *Machine Learning*, 24(1):49–64, 1996.

[11] Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 511–518, 2001.

[12] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, pages 4765–4774, 2017.

[13] Donald B. Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976.